Prerequisiti tecnici · Il pre-corso

Perché questa pagina

La certificazione **Claude Certified Architect — Foundations** è pensata per chi costruisce software. Per questo i materiali ufficiali e le domande d'esame **danno per scontati** molti concetti che uno sviluppatore usa ogni giorno: cos'è un JSON, come funziona una richiesta HTTP, cosa fa Git, cos'è una variabile d'ambiente.

Il corso non li spiega mai. Se non li conosci, le domande diventano difficili non per Claude, ma per quello che c'è intorno: leggi "il tool restituisce un errore 429", "lo schema JSON ha un campo `enum`" o "l'hook riceve l'input su stdin" e perdi il filo.

Questa pagina è un **pre-corso**: raccoglie, in linguaggio semplice, i concetti di base che servono prima di iniziare i domini. Non serve saperli usare da esperto. Serve riconoscerli e capire cosa significano quando compaiono in uno scenario.

> **Attenzione:** senza queste basi il superamento dell'esame non è garantito, anche se studi bene tutti i domini. Leggi questa pagina per prima e torna qui ogni volta che un termine non ti è chiaro.

Per ogni concetto trovi tra parentesi i domini in cui compare più spesso (D1–D5).

---

1. Formati dei dati

JSON (D2, D4, D5)

JSON (JavaScript Object Notation) è il formato di testo più usato per scambiare dati tra programmi. Claude riceve e restituisce JSON quando usa gli strumenti (tool) e quando produce output strutturato.

```
{
  "nome": "Mario Rossi",
  "eta": 42,
  "cliente_premium": true,
  "ordini": ["A-101", "A-102"],
  "indirizzo": { "citta": "Roma", "cap": "00100" },
  "note": null
}
```

Cosa devi sapere:

- un **oggetto** è racchiuso tra `{ }` ed è fatto di coppie **chiave: valore**;
- un **array** (lista) è racchiuso tra `[ ]`;
- i tipi di valore sono: stringa (`"testo"`), numero (`42`), booleano (`true`/`false`), `null` (valore assente), oggetto e array;
- le chiavi vanno sempre tra virgolette doppie;
- un JSON **non valido** (virgola di troppo, virgolette mancanti) non può essere letto dal programma: è uno degli errori tipici dell'output generato da un modello.

JSON Schema (D2, D4)

Un **JSON Schema** è un JSON che descrive **come deve essere fatto** un altro JSON: quali campi esistono, di che tipo sono, quali sono obbligatori.

```
{
  "type": "object",
  "properties": {
    "categoria": { "type": "string", "enum": ["fattura", "reclamo", "altro"] },
    "importo": { "type": "number" }
  },
  "required": ["categoria"]
}
```

- `type` indica il tipo del campo;
- `properties` elenca i campi;
- `required` elenca i campi obbligatori;
- `enum` limita i valori ammessi a una lista chiusa.

È fondamentale per l'esame: i tool di Claude si definiscono con un `input_schema` in JSON Schema, e l'output strutturato si ottiene facendo rispettare uno schema. Ricorda la differenza: lo schema garantisce la **forma** dei dati, non che il contenuto sia **corretto**.

YAML e frontmatter (D3)

YAML è un altro formato di testo per dati, più leggibile del JSON: usa l'indentazione al posto delle parentesi.

```
name: revisore-codice
description: Rivede il codice e segnala problemi
tools:
  - Read
  - Grep
```

L'indentazione **conta**: uno spazio sbagliato cambia il significato o rompe il file.

Il **frontmatter** è un blocco YAML messo all'inizio di un file Markdown, tra due righe `---`. In Claude Code lo trovi nelle skill, nei comandi e nei subagenti: contiene i metadati (nome, descrizione, strumenti permessi), mentre il resto del file contiene le istruzioni.

```
---
name: revisore-codice
description: Rivede il codice e segnala problemi
---

Istruzioni in linguaggio naturale…
```

Markdown (D3, D4)

Markdown è un modo di formattare il testo con pochi simboli. Il file resta testo semplice, leggibile anche così com'è, ma i programmi che lo "interpretano" (GitHub, gli editor, questa stessa pagina) lo mostrano con titoli, elenchi e grassetti.

La sintassi essenziale:

````
# Titolo principale
## Sottotitolo
### Titolo di terzo livello

Testo normale, **grassetto**, *corsivo* e `codice in linea`.

- elemento di un elenco puntato
- altro elemento

1. primo passo
2. secondo passo

[testo del link](https://esempio.it)

> citazione o nota

```python
print("blocco di codice")
```
````

Perché conta per l'esame:

- `CLAUDE.md`, le **skill** (`SKILL.md`), i comandi personalizzati e i subagenti di Claude Code sono file Markdown. I titoli `#` e `##` servono a organizzare le istruzioni per argomento, così sono facili da leggere sia per le persone sia per il modello.
- Claude stesso risponde spesso in Markdown. Se l'output deve essere letto da un **programma** (non da una persona), il Markdown diventa un problema: asterischi e cancelletti sporcano il dato. In quei casi si chiede JSON o testo semplice.
- Il **blocco di codice** (tre backtick ```) serve a delimitare codice o dati dentro un testo, senza che vengano confusi con le istruzioni.

XML e tag (D4)

L'**XML** (eXtensible Markup Language) è un formato che racchiude ogni dato tra un **tag di apertura** e un **tag di chiusura** con lo stesso nome:

```
<ordine id="A-101">
  <cliente>Mario Rossi</cliente>
```

```
  <totale valuta="EUR">120.50</totale>
</ordine>
```

- `<cliente>` apre, `</cliente>` chiude;
- i tag si possono annidare uno dentro l'altro, come le cartelle;
- un tag può avere **attributi** (`id="A-101"`).

Come formato di scambio dati l'XML è stato in gran parte sostituito dal JSON, ma resta in molti sistemi aziendali più vecchi.

Per l'esame è importante soprattutto un altro uso: nel **prompt engineering** con Claude si usano **tag in stile XML** per separare le parti del prompt. Non serve un XML formalmente valido e i nomi dei tag li scegli tu:

```
<istruzioni>
Riassumi il contratto in cinque punti.
</istruzioni>

<contratto>
…testo lungo del contratto…
</contratto>

<esempio>
1. Durata: 24 mesi…
</esempio>
```

Così il modello capisce con precisione dove finiscono le istruzioni e dove inizia il materiale da elaborare. Questo riduce il rischio che un testo dentro il documento venga scambiato per un'istruzione. È utile anche per chiedere a Claude di **mettere la risposta dentro un tag** (per esempio `<risposta>…</risposta>`), così il programma può estrarla facilmente.

CSV (D4, D5)

Un **CSV** (Comma-Separated Values, "valori separati da virgola") è una tabella salvata come testo semplice:

```
id,cliente,importo,data
1,Mario Rossi,120.50,2026-03-01
2,"Bianchi, Luca",80.00,2026-03-02
```

- la **prima riga** di solito contiene i nomi delle colonne (intestazione);
- ogni riga successiva è un **record**;
- i valori sono separati da un **delimitatore**: virgola, ma in Italia spesso punto e virgola, perché la virgola si usa nei decimali;
- se un valore contiene il delimitatore, va racchiuso tra virgolette (`"Bianchi, Luca"`).

Problemi tipici: delimitatori diversi da quelli attesi, decimali con virgola o punto, date in formati diversi, caratteri accentati rovinati da una **codifica** sbagliata (per esempio UTF-8 letto come un'altra codifica).

Negli scenari d'esame il CSV compare come **input** (migliaia di righe da classificare, adatte all'elaborazione in batch) o come **output** di un'estrazione. Ricorda che un CSV non ha uno schema: niente impedisce una colonna mancante o un valore nel formato sbagliato. Per questo, quando l'affidabilità conta, si preferisce un output in JSON validato con uno schema.

---

2. Reti, API e comunicazione

Client e server (D2)

Un **server** è un programma che offre un servizio e resta in attesa di richieste. Un **client** è il programma che invia le richieste. Quando la tua applicazione chiama Claude, la tua applicazione è il client e i server di Anthropic sono il server. In MCP il ruolo è lo stesso: Claude Code fa da client, il server MCP espone strumenti e risorse.

Richiesta HTTP (D2, D5)

HTTP è il protocollo con cui i programmi comunicano sul web (HTTPS è la sua versione cifrata). Una richiesta HTTP contiene:

- un **metodo**: `GET` (leggi), `POST` (crea/invia), `PUT`/`PATCH` (modifica), `DELETE` (cancella);
- un **URL** (l'indirizzo, chiamato anche *endpoint*);
- gli **header**: informazioni aggiuntive, come la chiave di autenticazione o il formato dei dati;
- un **body** (corpo), spesso in JSON, con i dati inviati.

Il server risponde con un **codice di stato** e, di solito, un body. I codici da riconoscere:

Codice	Significato	Cosa fare di solito
`200` OK	Richiesta riuscita; il body contiene il risultato	—
`201` Created	Risorsa creata (tipico di un `POST`); di norma il body contiene la risorsa appena creata e l'header `Location` indica dove trovarla	—
`204` No Content	Richiesta riuscita, ma **nessun body** (tipico di un `DELETE`)	Non cercare dati nella risposta
`400` Bad Request	Richiesta sbagliata (dati mancanti o non validi)	Correggere la richiesta, ripetere non serve
`401` / `403`	Non autenticato / non autorizzato	Controllare credenziali e permessi
`404` Not Found	Risorsa non trovata	Verificare l'identificativo
`409` Conflict	Conflitto con lo stato attuale (es. risorsa già esistente)	Rileggere lo stato e decidere
`429` Too Many Requests	Troppe richieste (rate limit)	Aspettare e riprovare

Codice	Significato	Cosa fare di solito
`500` / `503`	Errore o indisponibilità del server	Errore temporaneo: riprovare più tardi

La regola generale: i codici **2xx** sono successi, i **4xx** sono errori di chi fa la richiesta, i **5xx** sono errori del server. Questa tabella aiuta molto nel D2: distinguere un errore **temporaneo** (si può riprovare) da uno **permanente** (riprovare è inutile) è un tema ricorrente.

API e REST (D2)

Un'**API** (Application Programming Interface) è un "contratto" con cui un programma mette a disposizione alcune funzioni ad altri programmi, senza mostrare come funziona all'interno. Pensa al menu di un ristorante: sai cosa puoi ordinare e cosa ti arriverà, non come lavora la cucina. La **Claude API** è l'API con cui la tua applicazione invia messaggi a Claude e riceve le risposte.

Un'API **REST** è uno stile molto diffuso di API basata su HTTP, con alcune regole:

- ogni **risorsa** ha un indirizzo: `/clienti`, `/clienti/42`, `/clienti/42/ordini`;
- il **metodo** HTTP indica l'azione: `GET /clienti/42` legge il cliente 42, `POST /clienti` ne crea uno nuovo, `PATCH /clienti/42` lo modifica, `DELETE /clienti/42` lo cancella;
- i dati viaggiano di solito in JSON;
- è **stateless** (senza stato): ogni richiesta contiene tutto ciò che serve, il server non "ricorda" le richieste precedenti.

Un esempio completo:

```
POST /v1/clienti HTTP/1.1
Host: api.esempio.it
Authorization: Bearer <chiave>
Content-Type: application/json

{ "nome": "Mario Rossi", "email": "mario@esempio.it" }
```

Risposta:

```
HTTP/1.1 201 Created
Location: /v1/clienti/42

{ "id": 42, "nome": "Mario Rossi", "email": "mario@esempio.it" }
```

Il legame con l'esame: molti **tool** e **server MCP** sono un "involucro" attorno a un'API esistente (il CRM, il gestionale degli ordini, GitHub). Progettare bene un tool significa anche decidere quali operazioni dell'API esporre, con quali parametri, e come restituire a Claude gli errori dell'API in modo comprensibile. Ricorda anche che la Claude API è **stateless**: a ogni chiamata devi inviare di nuovo tutta la conversazione.

Chiave API e autenticazione (D2, D3)

Una **chiave API** è una lunga stringa segreta che identifica chi fa la richiesta, come una password pensata per i programmi. Si invia in un header a ogni chiamata (per la Claude API è l'header `x-api-key`). Il servizio la usa per:

- **autenticare**: sapere chi sei;
- **autorizzare**: sapere cosa puoi fare;
- **contare e fatturare** l'uso e applicare i rate limit.

Chi possiede la chiave può usare il servizio **a tuo nome e a tue spese**. Per questo:

- non va mai scritta nel codice né in file salvati nel repository (anche un repository privato può diventare pubblico o essere copiato);
- va letta da una **variabile d'ambiente** o da un **gestore di segreti** (secret manager, i "secrets" della CI);
- si danno chiavi diverse ad ambienti diversi (sviluppo, produzione), così se una viene rubata si può **revocare** senza fermare tutto;
- va **ruotata** (sostituita) periodicamente o subito se è stata esposta.

Termini collegati:

- **Token** (di accesso): simile a una chiave, ma di solito ha una **scadenza** e permessi limitati. Attenzione: non va confuso con i token di testo di un modello linguistico.
- **Bearer token**: un token inviato nell'header `Authorization: Bearer <token>`; "bearer" significa che chiunque lo possieda può usarlo.
- **OAuth**: un sistema in cui l'utente concede a un'applicazione l'accesso ai propri dati (per esempio "consenti a questa app di leggere il tuo calendario") senza consegnarle la password. L'applicazione riceve un token con permessi limitati e revocabili. Molti server MCP remoti usano OAuth.

Rate limit, timeout, retry e backoff (D2, D5)

- **Rate limit**: il limite di richieste (o di token) che puoi usare in un certo tempo. Superarlo produce l'errore `429`.
- **Timeout**: il tempo massimo di attesa di una risposta; scaduto quello, la richiesta viene considerata fallita.
- **Retry**: ripetere automaticamente una richiesta fallita. Ha senso solo per errori temporanei.
- **Backoff esponenziale**: tra un retry e l'altro si aspetta sempre di più (1 s, 2 s, 4 s…) per non sovraccaricare il servizio.
- **Latenza**: il tempo che passa tra la richiesta e la risposta.

Idempotenza (D2, D5)

Un'operazione è **idempotente** se eseguirla una o più volte produce lo stesso risultato. Leggere un ordine è idempotente; "addebita 50 €" no: ripeterla per errore addebita due volte. Prima di fare retry automatici su un'azione, chiediti sempre se è idempotente.

Webhook (D2, D5)

Normalmente è il tuo programma che chiede: "ci sono novità?". Se lo fa a intervalli regolari si chiama **polling**, ed è uno spreco: la maggior parte delle volte la risposta è "no".

Un **webhook** rovescia la situazione: sei tu a dare al servizio esterno un tuo indirizzo (URL), e **il servizio ti chiama** con una richiesta HTTP `POST` quando succede qualcosa. Esempi:

- il sistema di pagamenti avvisa "pagamento ricevuto per l'ordine A-101";
- GitHub avvisa "è stata aperta una nuova pull request";
- il sistema di ticket avvisa "nuovo ticket del cliente 42".

```
{
  "evento": "ticket.creato",
  "ticket_id": "T-9001",
  "cliente_id": 42,
  "timestamp": "2026-03-01T10:15:00Z"
}
```

Cose da sapere:

- chi riceve il webhook deve **rispondere in fretta** (con un `200`), altrimenti il mittente considera la consegna fallita e la ripete. Per questo il lavoro lungo (per esempio far analizzare il ticket a Claude) si mette in **coda** e si fa dopo;
- lo stesso evento può arrivare **più volte**: l'elaborazione deve essere idempotente (per esempio controllando se quel `ticket_id` è già stato gestito);
- il ricevente deve **verificare** che la chiamata venga davvero dal servizio atteso (di solito con una firma o un segreto condiviso).

Negli scenari d'esame un webhook è spesso il "grilletto" che fa partire un agente o una pipeline.

Streaming (D5)

Lo **streaming** è ricevere una risposta a pezzi, mentre viene generata, invece di aspettarla tutta. Con Claude si usa per mostrare il testo all'utente man mano che arriva: la risposta non è più veloce, ma l'utente vede subito qualcosa.

Sincrono, asincrono e batch (D4, D5)

Sincrono: il programma invia la richiesta e **resta in attesa** finché non arriva la risposta. È come una telefonata: finché non ti rispondono non fai altro. È il modo normale di chiamare la Claude API ed è necessario quando una persona aspetta la risposta (una chat, un assistente clienti).

Asincrono: il programma invia la richiesta e **continua a fare altro**; il risultato arriva più tardi, e il programma lo raccoglie quando è pronto (controllando periodicamente o ricevendo un avviso, per esempio un webhook). È come mandare un'email: non resti davanti allo schermo ad aspettare.

All'interno di un programma, "asincrono" indica anche la possibilità di lanciare **più operazioni insieme** (per esempio dieci chiamate a tool indipendenti) invece di aspettarle una alla volta.

Batch: si raggruppano **molte richieste** in un unico invio, che il sistema elabora con calma. Con la **Message Batches API** di Claude:

- i risultati arrivano entro un tempo massimo (fino a 24 ore, spesso molto prima);

- il costo è circa la **metà** rispetto alle chiamate normali;
- ogni richiesta ha un **identificativo personalizzato** (`custom_id`) per abbinare ogni risultato alla sua richiesta, perché l'ordine dei risultati non è garantito;
- alcune richieste del batch possono fallire mentre le altre riescono: vanno controllate e, se serve, ripetute singolarmente.

La domanda da farsi negli scenari d'esame è: **qualcuno sta aspettando la risposta?**

Situazione	Scelta tipica
Utente in chat, controllo bloccante prima di unire una PR	Sincrono
Analisi notturna di 50.000 documenti, report settimanale	Batch
Lavoro lungo avviato da un evento, senza persona in attesa	Asincrono (coda) o batch

Il batch **non** va bene quando serve una conversazione a più turni o un ciclo con i tool, perché ogni richiesta del batch è indipendente e non puoi rispondere in mezzo.

---

3. Riga di comando e sistema operativo

Terminale, shell e CLI (D3)

Il **terminale** è la finestra in cui si scrivono comandi testuali; la **shell** (bash, zsh, PowerShell) è il programma che li interpreta. Una **CLI** (Command Line Interface) è un programma che si usa dal terminale: Claude Code è una CLI e si avvia con il comando `claude`.

Argomenti e flag (D3)

I comandi accettano **argomenti** e **flag** (opzioni che iniziano con `-` o `--`). Esempio: `claude -p "Rivedi questo file" --output-format json` esegue Claude Code in modalità non interattiva (`-p`) e chiede l'output in JSON. Molte domande sulla CI/CD si basano proprio su questi flag.

stdin, stdout, stderr ed exit code (D3)

Ogni programma da riga di comando ha tre "canali":

- **stdin** (standard input): da dove legge i dati in ingresso;
- **stdout** (standard output): dove scrive il risultato normale;
- **stderr** (standard error): dove scrive messaggi di errore.

Quando termina, restituisce un **exit code**: `0` significa successo, un valore diverso da zero significa errore. Gli **hook** di Claude Code funzionano così: ricevono un JSON su stdin e comunicano l'esito con l'exit code (per esempio, un certo codice blocca l'azione) e con quello che scrivono su stdout/stderr.

Pipe e redirect (D3)

La **pipe** `|` passa l'output di un comando come input al successivo: `cat log.txt | claude -p "riassumi"`. Il **redirect** `>` salva l'output in un file: `comando > risultato.txt`.

Variabili d'ambiente (D2, D3)

Una **variabile d'ambiente** è un valore impostato fuori dal codice e letto dai programmi in esecuzione, per esempio `ANTHROPIC_API_KEY`. Si usano per configurazioni e segreti, così lo stesso codice funziona su macchine diverse senza scrivere password nei file. Nella configurazione dei server MCP si fa spesso riferimento a variabili d'ambiente come `${GITHUB_TOKEN}`.

Processo, subprocess e stdio (D2)

Un **processo** è un programma in esecuzione. Un **subprocess** è un processo avviato da un altro processo. Un server MCP con trasporto **stdio** è proprio questo: Claude Code lo avvia come subprocess sulla tua macchina e comunica con lui tramite stdin/stdout. Un server MCP **remoto**, invece, si raggiunge via HTTP.

File system, percorsi e glob (D3)

- Un **percorso** (path) indica dove si trova un file: **assoluto** se parte dalla radice (`/home/mario/progetto/app.py`), **relativo** se parte dalla cartella corrente (`./src/app.py`).
- La **home** dell'utente (`~`) è la cartella personale: lì vivono le configurazioni a livello utente, come `~/.claude/`.
- Un **pattern glob** seleziona più file con caratteri jolly: `*.py` (tutti i file Python nella cartella), `src/**/*.ts` (tutti i file TypeScript dentro `src`, a qualunque profondità). Le regole di Claude Code per percorso usano i glob.

Regex (D2, D3, D5)

Un'**espressione regolare** (regex) è un piccolo linguaggio per descrivere **schemi di testo** e trovare, controllare o estrarre le parti che li rispettano. È molto più potente di un semplice "cerca parola".

I mattoni principali:

Simbolo	Significato	Esempio
`\d`	una cifra	`\d\d` trova "42"
`\w`	una lettera, cifra o `_`	
`\s`	uno spazio	
`.`	qualunque carattere	
`*` / `+` / `?`	zero o più / uno o più / zero o uno	`\d+` trova "2026"
`{n}`	esattamente n volte	`\d{5}` trova un CAP
`[abc]`	uno dei caratteri indicati	`[A-Z]` una lettera maiuscola
`^` / `$`	inizio / fine del testo	`^\d{5}$` solo un CAP, nient'altro
`|`	oppure	`gatto|cane`
`( )`	gruppo (per estrarre una parte)	

Esempi pratici:

- `^[\w.-]+@[\w-]+\.[a-z]{2,}$` controlla, in modo semplificato, che un testo abbia la forma di un'email;

- `FATT-\d{4}-\d{6}` trova numeri di fattura come `FATT-2026-000123`;
- `\b\d{16}\b` trova sequenze di 16 cifre, cioè possibili numeri di carta da mascherare.

Perché conta per l'esame: la regex è **deterministica**. Con lo stesso testo dà sempre lo stesso risultato, al contrario di un modello linguistico. Per questo è lo strumento giusto per i controlli che devono essere **garantiti**:

- in un **hook**, per bloccare un comando pericoloso (per esempio qualunque comando che contiene `rm -rf`) o per impedire la scrittura in certi file;
- per **validare** un dato estratto da Claude (il codice fiscale ha la forma giusta? la data è nel formato atteso?);
- per trovare e mascherare dati sensibili prima di inviarli al modello.

I suoi limiti: controlla la **forma**, non il **significato**. Una regex sa dire che `31/02/2026` ha la forma di una data, non che quel giorno non esiste. E non capisce le frasi: per classificare il tono di un'email serve il modello, non una regex. Inoltre le regex complesse diventano difficili da leggere e fragili; meglio tenerle semplici e testarle.

Permessi (D3)

Un **permesso** stabilisce cosa un utente o un programma **può** fare. Esistono a più livelli.

Nel sistema operativo, ogni file ha permessi di **lettura**, **scrittura** ed **esecuzione** per il proprietario, il gruppo e gli altri. Un programma eseguito dal tuo utente può fare tutto quello che puoi fare tu: per questo un agente che esegue comandi sulla tua macchina va controllato.

Nei servizi online, una chiave o un token hanno un insieme di permessi (gli **scope**): per esempio "solo lettura dei repository", oppure "lettura e scrittura". Un utente può avere un **ruolo** (amministratore, editor, lettore) che determina cosa può fare.

In Claude Code, i permessi decidono quali strumenti Claude può usare e come:

- regole **allow** (consentito senza chiedere), **ask** (chiedi conferma) e **deny** (vietato);
- regole specifiche, per esempio consentire `Bash(npm test)` ma non qualunque comando;
- **modalità di permesso**, che vanno dal chiedere conferma per ogni modifica all'accettare automaticamente le modifiche ai file;
- i permessi si configurano nei file `settings.json`, a livello di utente, di progetto o gestiti dall'organizzazione.

Due principi da ricordare per l'esame:

- **Privilegio minimo**: concedi solo i permessi strettamente necessari al compito. Un subagente che deve solo leggere il codice non ha bisogno dello strumento per scrivere file né di eseguire comandi.
- **I permessi sono controlli deterministici**: una regola `deny` blocca davvero l'azione, mentre scrivere nel prompt "non modificare questi file" è solo una richiesta che il modello potrebbe non rispettare.

---

4. Sviluppo software e collaborazione

Repository e Git (D3)

Git è un sistema di **controllo di versione**: registra ogni modifica fatta ai file di un progetto, chi l'ha fatta e quando, e permette di tornare indietro o lavorare in parallelo senza pestarsi i piedi. Un **repository** (repo) è la cartella di progetto gestita da Git, con tutta la sua storia.

Il ciclo di lavoro tipico:

```
git clone https://github.com/azienda/progetto.git   # copia il repository in locale
git checkout -b correzione-login                    # crea un nuovo branch e ci si sposta
# …modifichi i file…
git status                                          # cosa è cambiato?
git diff                                            # mostra le differenze riga per riga
git add app.py                                      # prepara le modifiche per il salvataggio
git commit -m "Corregge il controllo della password" # salva una "fotografia"
git push                                            # invia i commit al repository remoto
```

I concetti chiave:

- **commit**: una "fotografia" delle modifiche, con un messaggio che le descrive e un identificativo unico (hash). La storia del progetto è una sequenza di commit;
- **branch**: una linea di sviluppo parallela. Il branch principale si chiama di solito `main`; per ogni modifica si crea un branch separato, così `main` resta stabile;
- **merge**: unire le modifiche di un branch in un altro. Se due persone hanno cambiato le stesse righe nasce un **conflitto**, da risolvere a mano;
- **diff**: l'elenco delle differenze tra due versioni, riga per riga. È quello che si mostra a Claude per una code review;
- **remoto** (remote): la copia del repository su un server (per esempio GitHub). `push` invia, `pull` scarica;
- `.gitignore`: il file che elenca cosa Git non deve tracciare (file temporanei, dipendenze scaricate, file con segreti);
- **worktree**: una seconda cartella di lavoro collegata allo stesso repository, su un altro branch. Permette di far lavorare più sessioni (o più agenti) in parallelo senza che si disturbino.

Nel D3 è importante capire **cosa finisce nel repository** e quindi è condiviso con tutto il team (`.claude/settings.json`, il `CLAUDE.md` di progetto, i comandi in `.claude/commands/`, il file `.mcp.json`) e **cosa resta solo sulla tua macchina** (`settings.local.json`, le impostazioni in `~/.claude/`). Una regola che deve valere per tutti va messa nei file condivisi; una preferenza personale no.

GitHub (D3)

GitHub è il servizio online più diffuso per ospitare repository Git. Aggiunge a Git gli strumenti per collaborare:

- **pull request** (PR): la proposta di unire un branch in `main`. Mostra il diff, permette di commentare singole righe, richiede l'approvazione dei revisori e lancia i controlli automatici prima del merge;
- **code review**: la revisione delle modifiche da parte di un collega (o di Claude) all'interno della PR;
- **issue**: segnalazioni di bug, richieste di funzionalità o attività da fare, con discussione e assegnatari;

- **GitHub Actions**: il sistema di automazione di GitHub. Si configura con file YAML nella cartella `.github/workflows/` e fa partire una sequenza di passaggi quando succede un evento (una PR aperta, un push, un commento);
- **secrets**: valori segreti (come una chiave API) salvati nelle impostazioni del repository e resi disponibili alle Actions come variabili d'ambiente, senza scriverli nel codice.

Claude Code si integra con GitHub: può essere richiamato in una PR o in una issue, revisionare il codice e proporre modifiche. Alternative simili a GitHub sono GitLab e Bitbucket.

CI/CD e pipeline (D3)

CI (Continuous Integration, integrazione continua) significa che a ogni modifica proposta un sistema automatico **verifica** che il codice funzioni ancora: lo compila, esegue i test, controlla lo stile. Lo scopo è scoprire subito gli errori, quando sono piccoli e facili da correggere.

CD ha due significati vicini: **Continuous Delivery** (il codice verificato è sempre pronto per essere rilasciato, con un clic) e **Continuous Deployment** (il rilascio in produzione avviene in automatico se tutti i controlli passano).

Una **pipeline** è la sequenza di passaggi automatici. Un esempio con GitHub Actions:

```yaml
name: revisione
on: pull_request          # parte a ogni pull request
jobs:
  controlli:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4      # scarica il codice
      - run: npm install               # installa le dipendenze
      - run: npm test                  # esegue i test
      - run: claude -p "Rivedi il diff e segnala problemi" --output-format json
        env:
          ANTHROPIC_API_KEY: ${{ secrets.ANTHROPIC_API_KEY }}
```

Cose da sapere per l'esame:

- in una pipeline **non c'è nessuna persona davanti allo schermo**: Claude Code va eseguito in modalità **non interattiva** (`-p`), perché nessuno risponderebbe a una domanda o a una richiesta di permesso, e la pipeline resterebbe bloccata;
- l'output deve essere **leggibile da un programma** (per esempio JSON, con `--output-format json`), così un passaggio successivo può pubblicare i commenti nella PR o decidere se fermare il rilascio;
- ogni passaggio comunica il successo o il fallimento con l'**exit code**: un exit code diverso da zero ferma la pipeline;
- i **segreti** arrivano dai secrets della piattaforma, mai dal codice;
- gli strumenti consentiti vanno limitati al minimo: una revisione automatica non deve poter modificare il codice o fare push;
- i costi e i tempi contano: una pipeline gira molte volte al giorno.

Test, lint e code review (D3)

- **Test**: codice che verifica automaticamente che altro codice funzioni. I **test unitari** controllano un singolo pezzo isolato; i **test di integrazione** controllano più parti insieme.
- **Lint/linter**: strumento che controlla stile ed errori comuni senza eseguire il codice.
- **Code review**: revisione del codice da parte di qualcun altro prima dell'unione.

Librerie, SDK e pacchetti (D1, D2)

Una **libreria** è codice già scritto da altri che puoi riusare nel tuo programma invece di riscriverlo: per leggere i PDF, fare richieste HTTP, gestire le date. Il tuo programma la "importa" e ne usa le funzioni.

Un **framework** è simile, ma più invasivo: non sei tu a chiamare lui quando serve, è lui a fornire la struttura generale e a chiamare il tuo codice nei punti previsti.

Un **SDK** (Software Development Kit) è un insieme di librerie, strumenti ed esempi per lavorare con una certa piattaforma. Per Claude ce ne sono due che devi distinguere bene:

- il **Claude SDK** (client SDK, per esempio `anthropic` in Python o `@anthropic-ai/sdk` in TypeScript): semplifica le chiamate alla Claude API. Invece di costruire a mano la richiesta HTTP, scrivi `client.messages.create(...)`. Il ciclo con i tool lo gestisci tu;
- il **Claude Agent SDK**: permette di costruire agenti con le stesse capacità di Claude Code (ciclo agentico già pronto, strumenti per file e comandi, subagenti, hook, MCP, gestione del contesto).

```
import anthropic

client = anthropic.Anthropic()          # legge la chiave da ANTHROPIC_API_KEY
risposta = client.messages.create(
    model="claude-…",
    max_tokens=1024,
    messages=[{"role": "user", "content": "Ciao!"}],
)
print(risposta.content[0].text)
```

Un **pacchetto** è una libreria confezionata per essere installata. Il **package manager** la scarica e la installa insieme alle sue **dipendenze** (le altre librerie di cui ha bisogno):

- `pip` (o `uv`) per Python, `npm` per JavaScript/TypeScript;
- `npx` (JavaScript) e `uvx` (Python) scaricano ed eseguono un pacchetto al volo, senza installarlo in modo permanente: per questo compaiono spesso nella configurazione dei server MCP, per esempio `"command": "npx", "args": ["-y", "nome-server-mcp"]`.

Una **versione** di un pacchetto si indica di solito come `2.4.1` (versione principale, minore, correzione). Fissare la versione evita che un aggiornamento cambi il comportamento del sistema senza che tu lo sappia.

Funzioni, parametri e valori di ritorno (D2)

Una **funzione** è un blocco di codice con un nome, che fa un compito preciso. Riceve dei **parametri** (gli input), esegue delle istruzioni e restituisce un **valore di ritorno** (l'output). Si scrive una volta e si "chiama" tutte le volte che serve.

```
def cerca_ordine(ordine_id: str, includi_righe: bool = False) -> dict:
    """Restituisce lo stato di un ordine dato il suo identificativo."""
    ordine = database.leggi("ordini", ordine_id)
    if ordine is None:
        raise ValueError(f"Ordine {ordine_id} non trovato")
    return {"id": ordine_id, "stato": ordine.stato}
```

Elementi da riconoscere:

- il **nome** (`cerca_ordine`) dice cosa fa;
- i **parametri** hanno un nome e un **tipo** (`str`, `bool`); alcuni sono **obbligatori**, altri **facoltativi** con un valore predefinito (`includi_righe = False`);
- la **docstring** (il testo tra virgolette) descrive la funzione;
- il **valore di ritorno** è il risultato;
- un'**eccezione** (`raise`) segnala che qualcosa è andato storto, invece di restituire un risultato.

Il collegamento con l'esame è diretto: un **tool** di Claude è la descrizione di una funzione.

Funzione nel codice	Tool per Claude
nome della funzione	`name`
docstring	`description`
parametri, tipi, obbligatori	`input_schema` (JSON Schema con `properties` e `required`)
valore di ritorno	contenuto del `tool_result`
eccezione	`tool_result` con `is_error: true` e un messaggio chiaro

La differenza fondamentale: **Claude non esegue la funzione**. Claude chiede di chiamarla (blocco `tool_use` con nome e argomenti), la tua applicazione la esegue davvero e restituisce il risultato a Claude. Per questo la **descrizione** è così importante: è l'unica cosa che Claude legge per decidere quando e come usare il tool.

Cicli e condizioni (D1)

Una **condizione** fa scegliere al programma tra strade diverse: "**se** succede questo, fai A, **altrimenti** fai B".

```
if importo > 1000:
    richiedi_approvazione_umana()
else:
    procedi_con_il_rimborso()
```

Un **ciclo** ripete un blocco di istruzioni:

- `for` lo ripete per ogni elemento di una lista ("per ogni documento, estrai i dati");
- `while` lo ripete **finché** una condizione resta vera.

Ogni ciclo `while` ha bisogno di una **condizione di uscita**: se non diventa mai falsa, il programma gira all'infinito (**ciclo infinito**). Per sicurezza si aggiunge spesso anche un **limite massimo** di ripetizioni.

Il **ciclo agentico**, cuore del D1, è esattamente un ciclo `while` con delle condizioni:

```
messaggi = [{"role": "user", "content": richiesta}]
for giro in range(20):                      # limite di sicurezza
    risposta = client.messages.create(model=..., tools=tools, messages=messaggi)
    messaggi.append({"role": "assistant", "content": risposta.content})
    if risposta.stop_reason != "tool_use":   # condizione di uscita
        break                                # Claude ha finito
    risultati = esegui_i_tool(risposta.content)
    messaggi.append({"role": "user", "content": risultati})
```

Da notare:

- la decisione di **continuare o fermarsi** si prende leggendo `stop_reason`, un campo strutturato, non cercando parole come "ho finito" nel testo della risposta;
- il **limite di giri** è una rete di sicurezza contro i cicli infiniti, non il modo normale di terminare;
- le **condizioni** nel codice (come `if importo > 1000`) sono il modo deterministico di imporre una regola, ad esempio fermare un rimborso troppo alto prima che venga eseguito.

Deterministico e probabilistico (D1, D4, D5)

Un sistema è **deterministico** quando, con lo stesso input, dà **sempre lo stesso output**. Il codice tradizionale funziona così: `2 + 2` fa sempre 4, una regola `if importo > 1000` scatta sempre oltre i 1000 euro, una regex trova sempre le stesse corrispondenze.

Un sistema è **probabilistico** quando l'output può **variare** anche con lo stesso input. Un modello linguistico funziona così: genera il testo scegliendo tra continuazioni più o meno probabili. Di solito segue le istruzioni, ma non c'è la **garanzia** che lo faccia sempre. Con mille richieste, qualche caso andrà diversamente.

Non è un difetto: è proprio quello che permette al modello di capire testi ambigui, ragionare e adattarsi a situazioni nuove. Ma va usato nel posto giusto.

È probabilmente **l'idea più importante dell'intero esame**. La regola pratica:

Va affidato al codice (deterministico)	Va affidato al modello (probabilistico)
Regole che devono valere **sempre** (limiti di importo, permessi, conformità)	Capire una richiesta scritta in modo libero
Ordine obbligato dei passaggi (verifica l'identità **prima** del rimborso)	Classificare, riassumere, estrarre da testo non strutturato
Controlli di formato e validazione	Decidere quale strumento usare e in che ordine
Calcoli esatti	Gestire casi nuovi o ambigui

Esempi di traduzione in pratica:

- "Il rimborso sopra i 500 € deve essere approvato da una persona" → un **hook** o una condizione nel codice che blocca il tool, non una frase nel prompt;
- "L'output deve essere un JSON con questi campi" → **output strutturato** con uno schema, non solo "rispondi in JSON";
- "Claude non deve mai cancellare file" → una regola **deny** nei permessi, non un'istruzione.

Quando una domanda d'esame contiene parole come "sempre", "mai", "garantire", "obbligatorio", "conformità", la risposta corretta è quasi sempre quella **deterministica**. Le istruzioni nel prompt **aumentano la probabilità** di un comportamento; il codice lo **garantisce**.

---

5. Dati e archiviazione

Database (D2, D5)

Un **database** è un sistema che conserva grandi quantità di dati in modo organizzato e permette di **cercarli, aggiungerli, modificarli e cancellarli** in modo rapido e sicuro, anche con molti utenti contemporaneamente. Il programma che lo gestisce si chiama **DBMS** (per esempio PostgreSQL, MySQL, SQL Server, MongoDB).

Le quattro operazioni di base si riassumono con la sigla **CRUD**: **C**reate (crea), **R**ead (leggi), **U**pdate (aggiorna), **D**elete (cancella).

Database relazionali e SQL (D2, D5)

Un database **relazionale** organizza i dati in **tabelle**, come fogli di calcolo molto rigorosi:

Tabella `clienti`

id	nome	email
42	Mario Rossi	`mario@esempio.it`
43	Luca Bianchi	`luca@esempio.it`

Tabella `ordini`

id	cliente_id	totale	stato
A-101	42	120.50	spedito
A-102	42	80.00	in lavorazione

I concetti chiave:

- una **riga** (record) è un singolo elemento, una **colonna** (campo) è una sua caratteristica;
- ogni tabella ha uno **schema**: le colonne e il loro tipo (testo, numero, data) sono fissati in anticipo;
- la **chiave primaria** è la colonna che identifica in modo univoco ogni riga (`id`);
- la **chiave esterna** collega due tabelle: `ordini.cliente_id` punta a `clienti.id`. È la "relazione" che dà il nome a questi database;
- un **indice** rende veloci le ricerche su una colonna, come l'indice analitico di un libro.

Si interrogano con il linguaggio **SQL** (Structured Query Language). Una **query** è una richiesta al database:

```sql
-- leggi gli ordini del cliente 42 ancora da spedire
SELECT id, totale, stato
FROM ordini
WHERE cliente_id = 42 AND stato <> 'spedito';

-- unisci due tabelle (JOIN): ordini con il nome del cliente
SELECT o.id, c.nome, o.totale
FROM ordini o
JOIN clienti c ON c.id = o.cliente_id;

-- modifica e inserimento
UPDATE ordini SET stato = 'annullato' WHERE id = 'A-102';
INSERT INTO clienti (id, nome, email) VALUES (44, 'Anna Verdi',
'anna@esempio.it');
```

Senza `WHERE`, un `UPDATE` o un `DELETE` agisce su **tutte** le righe della tabella: è uno degli errori più gravi e più classici.

Database non relazionali (NoSQL) (D2, D5)

I database **NoSQL** non usano tabelle rigide. Ne esistono vari tipi:

- **a documenti** (MongoDB): ogni elemento è un documento simile a un JSON, che può avere campi diversi dagli altri;
- **chiave-valore** (Redis): come un grande dizionario, velocissimo, usato spesso come cache;
- **vettoriali**: conservano embedding per la ricerca per significato (vedi la sezione 6).

Sono più flessibili, ma spostano il controllo della struttura dei dati sull'applicazione.

Transazioni e integrità (D5)

Una **transazione** è un gruppo di operazioni che devono riuscire **tutte insieme o nessuna**. Esempio: in un bonifico si toglie il denaro da un conto e lo si aggiunge a un altro; se la seconda operazione fallisce, anche la prima viene annullata (**rollback**). Senza transazione resteresti con il denaro sparito.

Database e agenti: cosa conta per l'esame (D2, D5)

Negli scenari d'esame il database è spesso il "sistema backend" (il gestionale degli ordini, il CRM) a cui Claude accede tramite un tool o un server MCP. Le buone pratiche:

- **non dare a Claude un tool "esegui qualunque SQL"**: meglio tool specifici e limitati (`cerca_ordine`, `aggiorna_indirizzo`), con parametri validati. Sono più sicuri e Claude li usa meglio;
- separare i tool di **lettura** da quelli di **scrittura**, e usare credenziali di sola lettura dove basta leggere (privilegio minimo);
- le operazioni di scrittura importanti (rimborsi, cancellazioni) vanno **controllate dal codice** e, se serve, approvate da una persona;
- restituire **solo i campi necessari** e **pochi risultati**: una query che restituisce 10.000 righe riempie la finestra di contesto e confonde il modello. Meglio filtri, limiti e paginazione;

- distinguere nel risultato del tool **"nessun risultato trovato"** (una risposta valida e vuota) da **"il database non ha risposto"** (un errore): sono situazioni diverse e Claude deve poterle riconoscere;
- non costruire le query incollando il testo prodotto dal modello o dall'utente: si usano **query parametrizzate**, per evitare la **SQL injection** (testo malevolo che modifica la query).

Cache (D5)

Una **cache** è una copia temporanea di dati usati spesso, per non doverli ricalcolare o riscaricare. Il **prompt caching** di Claude applica questa idea alla parte iniziale e ripetuta di un prompt: riduce costi e latenza.

Log e audit trail (D5)

Un **log** è il registro degli eventi di un programma. Un **audit trail** è una traccia di chi ha fatto cosa e quando, utile per controlli e conformità. Nei sistemi affidabili si registrano decisioni, errori e passaggi agli operatori umani.

Dati personali e sensibili (D5)

PII (Personally Identifiable Information) sono i dati che identificano una persona: nome, email, codice fiscale, numero di carta. Vanno protetti, mascherati quando non servono e trattati secondo le norme sulla privacy (in Europa il **GDPR**).

---

6. Documenti e intelligenza artificiale

OCR (D4, D5)

L'**OCR** (Optical Character Recognition, riconoscimento ottico dei caratteri) trasforma l'**immagine** di un testo in **testo vero**, che un programma può cercare, copiare ed elaborare. Una fattura scansionata, a prima vista, è solo una fotografia: per il computer contiene pixel, non lettere. L'OCR riconosce le forme dei caratteri e ricostruisce le parole.

Gli errori tipici dell'OCR:

- **caratteri simili scambiati**: `0` e `O`, `1`, `l` e `I`, `5` e `S`, `rn` letto come `m`;
- **ordine sbagliato** del testo: in documenti a più colonne o con tabelle, le righe possono essere lette mescolate;
- **tabelle "appiattite"**: si perde l'informazione su quale valore appartiene a quale colonna;
- **qualità bassa**: scansioni storte, macchiate, a bassa risoluzione o scritte a mano producono testo pieno di errori;
- **elementi persi**: timbri, firme, caselle spuntate, note a margine.

Perché conta per l'esame: negli scenari di **estrazione dati** da documenti, un valore sbagliato può avere due cause molto diverse, che vanno distinte:

1. il **testo di partenza** era già sbagliato (errore dell'OCR): nessun prompt migliore lo correggerà;
2. il **modello** ha estratto male da un testo corretto: qui aiutano istruzioni, esempi e validazione.

Claude può anche leggere direttamente **immagini e PDF** (capacità di visione), senza un passaggio OCR separato, e in questo modo vede anche impaginazione e tabelle. In ogni caso, i dati estratti da documenti di bassa qualità vanno **validati** (somme che tornano, formati corretti) e, se incerti, passati a una persona.

PDF e documenti non strutturati (D4, D5)

Un dato **strutturato** ha una forma fissa e prevedibile: una tabella di database, un JSON con campi definiti. Un programma lo può leggere senza ambiguità.

Un dato **non strutturato** è testo libero o un documento pensato per le persone: email, contratti, verbali, chat, recensioni. L'informazione c'è, ma non sta in un posto preciso: la data di scadenza di un contratto può essere scritta in cento modi diversi, in qualunque pagina. Esistono anche dati **semi-strutturati** (moduli, fatture), con una struttura ricorrente ma non identica tra un documento e l'altro.

Il **PDF** è il formato più comune per i documenti. Ne esistono due tipi molto diversi:

- **PDF nativo** (generato da un programma): contiene il testo vero, che si può estrarre;
- **PDF scansionato**: contiene solo immagini delle pagine, e richiede OCR o lettura visiva.

Molti scenari d'esame chiedono di trasformare documenti non strutturati in dati strutturati. Lo schema tipico:

1. **acquisizione** del documento (ed eventuale OCR);
2. **estrazione** con Claude, chiedendo un output che rispetti uno **schema JSON**;
3. **validazione**: lo schema controlla la forma, il codice controlla la coerenza (le righe sommano al totale? la data è plausibile?);
4. **gestione dei casi incerti**: se un dato non è presente nel documento, il valore giusto è `null`, non un dato inventato; i casi dubbi vanno a una persona.

Per documenti molto lunghi bisogna anche gestire la finestra di contesto: dividere il documento in parti (**chunking**) o estrarre per sezioni.

Modello linguistico (LLM) (tutti i domini)

Un **LLM** (Large Language Model, modello linguistico di grandi dimensioni) è un modello di intelligenza artificiale addestrato su enormi quantità di testo. Il suo compito di base è **prevedere la continuazione più probabile** di un testo, un token alla volta. Da questo meccanismo semplice nascono capacità sorprendenti: rispondere a domande, ragionare, scrivere codice, seguire istruzioni. Claude è un LLM.

Cose da sapere:

- **non ha memoria tra una richiesta e l'altra**: a ogni chiamata riceve tutta la conversazione che gli invii (la API è stateless). Una "memoria" a lungo termine va costruita dall'esterno, salvando informazioni e reinserendole nel contesto;
- **ha una data limite delle conoscenze** (knowledge cutoff): non conosce i fatti successivi all'addestramento, a meno che tu non glieli fornisca, per esempio con un tool di ricerca;
- **non consulta un archivio di fatti**: genera testo plausibile. Per questo può sbagliare con sicurezza (vedi le allucinazioni);
- **può usare strumenti**: non esegue direttamente azioni nel mondo, ma può chiedere alla tua applicazione di eseguirle (i tool);
- **è probabilistico**: segue molto bene le istruzioni, ma senza garanzia assoluta.

Il **prompt** è tutto il testo che il modello riceve; un **modello** più grande è in genere più capace ma più lento e costoso, uno più piccolo è più veloce ed economico. Scegliere il modello giusto per ogni compito (per esempio uno piccolo per classificare, uno grande per ragionare) è una decisione di architettura.

Token e finestra di contesto (D5)

Un **token** è l'unità di testo che il modello elabora: un pezzo di parola, una parola intera, un segno di punteggiatura. In media, in italiano, una parola corrisponde a uno o due token. Tutto si misura in token:

- i **costi**: si pagano i token in ingresso (input) e quelli generati (output), questi ultimi più cari;
- i **limiti**: `max_tokens` fissa il massimo di token che il modello può generare in una risposta; se la risposta lo raggiunge, viene **troncata**;
- la **velocità**: più token da generare significa più tempo.

La **finestra di contesto** è la quantità massima di token che il modello può considerare in una singola richiesta. Ci deve stare **tutto**: prompt di sistema, definizioni dei tool, l'intera conversazione, i documenti, i risultati dei tool e la risposta.

Cosa succede in pratica:

- in una conversazione lunga o in un agente che usa molti tool, il contesto **cresce a ogni giro**;
- anche prima di essere pieno, un contesto molto lungo e disordinato rende il modello **meno preciso**: le informazioni importanti si "perdono" in mezzo a quelle inutili;
- quando si avvicina il limite bisogna **riassumere** (compattazione), **eliminare** i risultati dei tool vecchi, **selezionare** solo le parti utili o **dividere** il lavoro tra subagenti, ciascuno con un contesto pulito.

Un riassunto, però, perde i dettagli esatti: numeri, date e identificativi importanti vanno conservati a parte, in forma strutturata.

Temperatura (D4)

La **temperatura** è un parametro che regola **quanto il modello varia** nella scelta delle parole:

- **bassa** (vicina a 0): sceglie quasi sempre le continuazioni più probabili. Risposte più prevedibili e coerenti, adatte a estrazione, classificazione, codice;
- **alta**: dà più spazio a scelte meno probabili. Risposte più varie e creative, adatte a brainstorming e scrittura.

Attenzione: anche a temperatura 0 il modello **non diventa deterministico** in senso stretto e, soprattutto, una temperatura bassa **non rende le risposte più corrette**. Le rende solo più ripetibili: un errore può ripetersi uguale ogni volta. Nei modelli più recenti, inoltre, alcuni parametri di campionamento possono non essere modificabili; la qualità si governa soprattutto con istruzioni, esempi e struttura.

Allucinazioni (D4, D5)

Un'**allucinazione** è un'informazione **inventata** ma presentata come vera, con lo stesso tono sicuro di quelle corrette: una citazione inesistente, un numero di fattura plausibile ma sbagliato, una funzione di libreria che non esiste. Succede perché il modello genera testo plausibile, non controlla un archivio di fatti.

Sono più probabili quando:

- l'informazione **non è presente** nel contesto e il modello "riempie il vuoto";
- la domanda **presuppone** qualcosa di falso;
- il prompt spinge a dare **sempre** una risposta completa.

Come si riducono:

- fornire le **fonti** nel contesto e chiedere di rispondere **solo** sulla base di quelle;
- chiedere **citazioni** dei passaggi usati, così la risposta si può verificare;
- **permettere esplicitamente** di dire "non lo so" o di restituire `null` quando un dato manca: è la regola d'oro dell'estrazione dati;
- **validare** l'output con il codice (lo schema, i controlli di coerenza) e, per le decisioni importanti, con una persona.

Embedding, ricerca semantica e RAG (D5)

Un **embedding** è la trasformazione di un testo in una **lista di numeri** (un vettore), costruita in modo che testi con **significato simile** abbiano numeri **vicini**. "Come posso riavere i miei soldi?" e "procedura di rimborso" non hanno parole in comune, ma i loro embedding sono vicini.

La **ricerca semantica** usa gli embedding per trovare i testi più simili per **significato** a una domanda, e non solo quelli che contengono le stesse parole (come fa la ricerca per **parole chiave**). Le due tecniche sono complementari: la ricerca per parole chiave è migliore per codici esatti, nomi propri e numeri di pratica; quella semantica per le domande formulate liberamente. Spesso si combinano (**ricerca ibrida**).

Gli embedding si salvano in un **database vettoriale**, ottimizzato per trovare velocemente i vettori più vicini.

La **RAG** (Retrieval-Augmented Generation, generazione aumentata dal recupero) mette insieme questi pezzi per far rispondere il modello su documenti che non conosce (manuali aziendali, contratti, procedure):

1. **preparazione**: i documenti vengono divisi in pezzi (**chunk**) e per ognuno si calcola l'embedding;
2. **recupero**: alla domanda dell'utente si cercano i chunk più pertinenti;
3. **generazione**: i chunk trovati vengono inseriti nel prompt e il modello risponde basandosi su di essi, citandoli.

Vantaggi: risposte aggiornate e verificabili, senza dover inserire nel contesto l'intero archivio. Il punto debole: se il recupero porta i pezzi sbagliati, anche la risposta sarà sbagliata. Molti problemi di un sistema RAG sono problemi di **recupero**, non del modello.

Prompt di sistema e few-shot (D4)

Il **prompt di sistema** (system prompt) è la parte delle istruzioni che stabilisce il **contesto generale** per tutta la conversazione: il ruolo del modello ("sei l'assistente clienti di..."), il tono, le regole, il formato delle risposte. Nella Claude API si passa nel parametro `system`, separato dai messaggi dell'utente. È il posto giusto per le istruzioni stabili; i dati specifici della singola richiesta vanno invece nei messaggi.

Zero-shot significa chiedere un compito senza esempi. **Few-shot** significa includere nel prompt **alcuni esempi** di input con l'output desiderato:

```
<esempi>
<esempio>
<email>Il pacco è arrivato rotto, voglio i soldi indietro.</email>
<categoria>rimborso</categoria>
</esempio>
<esempio>
```

```
<email>Posso cambiare l'indirizzo di consegna?</email>
<categoria>spedizione</categoria>
</esempio>
</esempi>
```

Gli esempi sono spesso più efficaci delle spiegazioni, perché **mostrano** invece di descrivere. Sono particolarmente utili per:

- far rispettare un **formato** preciso;
- chiarire i **casi limite** e ambigui (come classificare un'email che chiede sia un rimborso sia un cambio di indirizzo);
- rendere coerenti i risultati tra una richiesta e l'altra.

Buone pratiche: esempi **vari** (non tutti uguali, altrimenti il modello copia dettagli irrilevanti), che coprano anche i **casi difficili**, e ben **separati** dal resto del prompt con dei tag.

---

7. Architettura e affidabilità

Architettura e componenti (D1)

L'**architettura** di un sistema è il modo in cui è diviso in componenti e come questi comunicano. Negli scenari d'esame i componenti tipici sono: l'interfaccia per l'utente, l'applicazione che chiama Claude, i tool, i server MCP, i sistemi backend (database, CRM, gestionali) e gli operatori umani.

Orchestrazione (D1)

Orchestrare significa coordinare più passaggi o più componenti verso un obiettivo. Un **coordinatore** (orchestrator) divide il lavoro e lo assegna; i **worker** (nel caso di Claude, i subagenti) eseguono le parti.

Parallelo e sequenziale (D1)

Operazioni **sequenziali** avvengono una dopo l'altra, perché ognuna dipende dalla precedente. Operazioni **parallele** avvengono contemporaneamente, perché sono indipendenti. Parallelizzare riduce i tempi, ma solo quando i compiti non dipendono uno dall'altro.

Fallimento parziale e degradazione controllata (D5)

In un sistema con molte parti, una può fallire mentre le altre funzionano. **Degradazione controllata** (graceful degradation) significa continuare a offrire un servizio ridotto, dichiarando cosa manca, invece di bloccarsi del tutto o, peggio, nascondere il problema.

Human-in-the-loop ed escalation (D1, D5)

Human-in-the-loop indica un sistema in cui una persona interviene in punti precisi: approva azioni rischiose, rivede casi incerti. L'**escalation** è il passaggio di un caso dall'automazione a un operatore umano, con tutte le informazioni necessarie per non ricominciare da capo.

SLA (D5)

Uno **SLA** (Service Level Agreement) è l'impegno sul livello di servizio, per esempio "risposta entro 24 ore". Negli scenari d'esame un SLA può decidere se usare elaborazione in tempo reale o in batch.

E adesso? Il vocabolario

Questa pagina spiega i concetti **generali** dell'informatica che il corso dà per scontati. I termini **specifici di Claude** (`stop_reason`, `tool_choice`, subagente, compattazione, hook, skill…) sono raccolti nel **Vocabolario**, in ordine alfabetico e con il dominio di riferimento.

Nel Vocabolario trovi anche una sezione **"Termini tecnici di base"** con una definizione breve di tutti i concetti di questa pagina, utile come ripasso veloce.

Il percorso consigliato:

1. leggi questo pre-corso fino in fondo;
2. tieni aperto il **Vocabolario** (lo trovi nel menu laterale) mentre studi;
3. inizia dal **Dominio 1** e, ogni volta che incontri un termine che non ti è chiaro, cercalo prima nel Vocabolario e poi qui.